

Claude Certified Architect Foundations (CCA-F) Master Cheat Sheet

Domain 1: Agentic Architecture and Orchestration (27%)

1.1 Designing Multi-Agent Systems

Definition

A **multi-agent system (MAS)** consists of multiple autonomous AI agents that interact, share context, and collaborate to achieve complex goals beyond a single agent's capability.

Key Concepts

- **Autonomy** – Each agent operates independently within its role.
- **Social ability** – Agents communicate via structured messages or shared memory.
- **Reactivity** – Agents perceive and respond to environment changes.
- **Proactiveness** – Agents take initiative toward goals.

Design Principles for CCA-F

Principle	Description
Single responsibility	Each agent handles one domain (e.g., SQL agent, email agent).
Shared schema	All agents agree on context format (e.g., JSON, tool definitions).
Idempotency	Repeating the same action yields same result (critical for retries).
Observability	Log agent decisions, tool calls, and state changes.

Example

- **Trip planning MAS:**
 - Flight agent, Hotel agent, Weather agent, Budget agent.
 - Orchestrator delegates subtasks → each returns structured data → final plan merged.

Practical Tip for Exam

Look for scenarios where **agents have different capabilities but share a common message bus or orchestrator**. Avoid monolithic prompts.

1.2. Task Decomposition

Definition

Breaking a complex user request into smaller, executable subtasks that can be assigned to specialized agents or tools.

Techniques

Technique	How it works	Exam focus
Top-down	Start with goal → break into logical steps (e.g., outline → research → write).	High
LLM-based decomposition	Use Claude to generate subtask list via chain of thought.	High
Plan-and-execute	Generate full plan first, then execute sequentially.	Medium
Dynamic decomposition	Re-plan if subtask fails or new info appears.	High for reliability

Example Prompt Pattern (Exam relevant)

User: "Analyze Q3 sales and email summary to stakeholders."

Decomposition:

1. Call SQL agent: fetch Q3 sales by region.
2. Call analytics agent: compute trend % change.
3. Call summarizer agent: convert numbers to bullet points.
4. Call email agent: draft email with summary + attach chart.

Key Takeaways

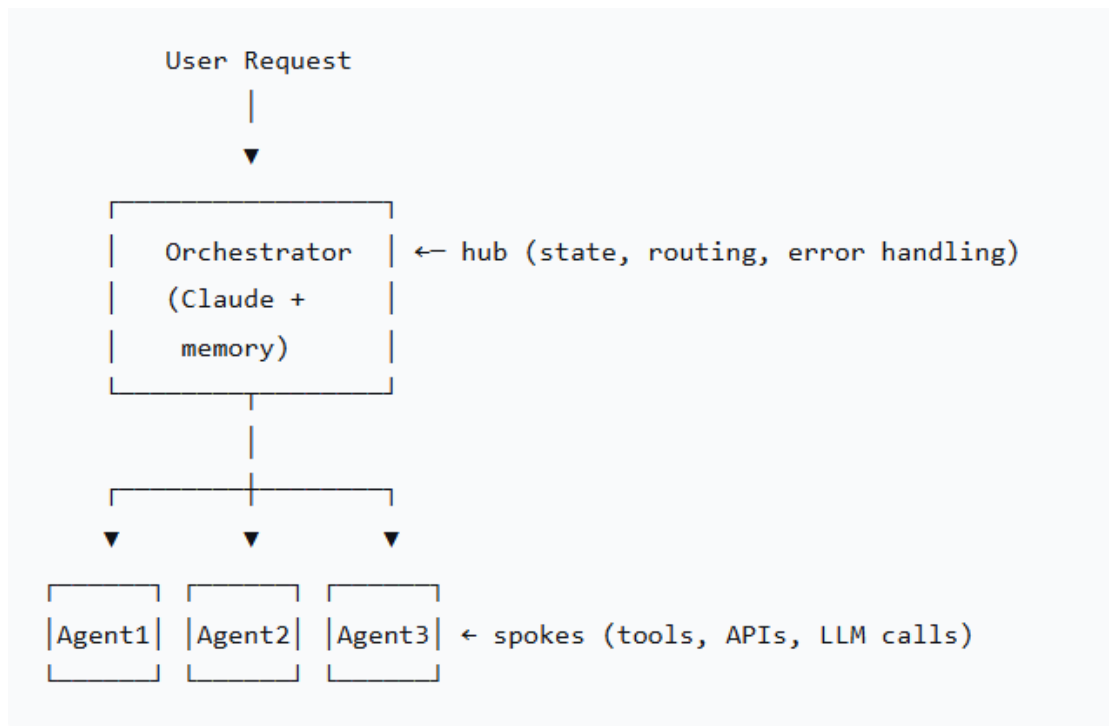
- Decomposition must preserve **data dependencies** (Task B needs Task A’s output).
- Each subtask should be **testable** and **small enough** for one agent/tool.

1.3. Hub-and-Spoke Orchestration Models

Definition

A central **orchestrator (hub)** receives user input, decomposes tasks, routes subtasks to specialized **agents (spokes)**, and aggregates results.

Diagram (Mental model for exam)



Key Characteristics

Feature	Hub-and-Spoke
Control	Centralized
State mgmt	Hub holds conversation state
Fault tolerance	Hub retries or re-routes
Scalability	Add spokes without changing logic
Latency	Higher (hub bottleneck)

When to Use (Exam Question Clue)

- ☒ Well-defined subtasks
- ☒ Need full audit trail
- ☒ Agents are stateless
- ☒ Real-time low-latency systems

Practical Implementation (CCA-F)

- Hub = Claude with **tool definitions** for each spoke agent.
- Spoke = Anthropic API call with system prompt specializing in one domain.

1.4. Agentic Loops

Definition

A cycle where an agent:

1. Observes current state
2. Thinks (plans next action)
3. Acts (calls tool / responds)
4. Observes result → repeats until goal met.

Loop Types

Type	Description	Exam importance
ReAct (Reason + Act)	Interleaves reasoning and tool use.	★ Core pattern
Reflection loop	Agent critiques own output and revises.	High for quality
While loop	Continue until condition (e.g., all rows processed).	Medium
Timeout loop	Exit if max iterations reached.	Critical for safety

Example (ReAct pattern)

User: "Book a flight to NYC under \$500"

Agent thinks: Need date, origin. Not given → ask.

Action: ask_user("What date and departure city?")

Observe: "March 10, Chicago"

Think: Now call flight API.

Action: search_flights(ORD→JFK, 2025-03-10, max=500)

Observe: No flights under \$500.

Think: Offer alternative dates.

Action: respond(...)

Key Takeaways for Exam

- Agentic loops **must have termination conditions** (max steps, timeout).
- Without loop, agent cannot handle multi-step tool use.

1.5. Subagent Coordination

Definition

Mechanisms by which multiple subagents communicate, share context, and avoid conflicts.

Coordination Patterns

Pattern	How it works	Exam scenario
Sequential	Agent A → Agent B → Agent C (pipeline)	Data transformation
Parallel	Run agents simultaneously, merge later	Speed-critical tasks
Hierarchical	Manager agent spawns worker agents	Complex breakdown
Broadcast	One agent publishes to all subscribers	Monitoring / logging

Shared Memory Approaches

Type	Example	Pros	Cons
Global dict	context["user_id"] = 123	Fast	Race conditions
Message queue	Redis / RabbitMQ	Reliable	Overhead
File store	JSON logs	Auditable	Slow
Vector DB	Chroma for RAG	Semantic search	Complex

Exam Tip

Coordination failures happen when agents **overwrite shared state** or **lack mutual exclusion**. Look for questions about “two agents trying to update same booking ID” – answer is *pessimistic locking* or *orchestrator token*.

We have 700 Practice test questions **Claude Certified Architect Foundations (CCA-F)** (Taken from previous exams)

Full Practice Set link below

<https://skillcertpro.com/product/claude-certified-architect-foundations-cca-f-exam-questions/>

100% Money back Guarantee, If you don't pass the exam in 1st attempt, your money will be refunded back

1.6. Reliability Patterns for Autonomous Systems

Key Reliability Risks in Agentic Systems

- Infinite loops
- Hallucinated tool calls
- Stuck in partial state
- Spoke agent unavailable
- Cost explosion (API calls)

Patterns to Know for Exam

Pattern	Description	Implementation (CCA-F)
Idempotency keys	Same request → same result, no duplicate effect	Store request_id and outcome
Circuit breaker	If agent fails 3+ times, skip for cooldown	Return cached/fallback response
Retry with backoff	Exponential delay between retries	Max 3–5 retries, jitter added
Timeouts per step	Kill loop after X seconds	Claude API timeout param
Human-in-the-loop	Escalate for high-stakes actions	Require approval before tool call
Checkpointing	Save state after each agentic step	Redis / DynamoDB session store
Validation agent	Secondary agent checks primary's output	Example: safety agent reviews email before send

Monitoring Metrics (Exam Memory Aid)

Latency per loop | Tool success rate | Retry frequency |
 Cost per session | State size | Escalation rate |

Example Reliability Flow

1. User request
2. Orchestrator starts loop (max 5 steps, timeout 30s)
3. Tool call fails → retry with exponential backoff (1s, 2s, 4s)
4. After 3 failures → circuit breaker opens → fallback response
5. Checkpoint saved after each successful step

6. On final response → validation agent reviews → if invalid, re-enter loop

Summary Table – Domain 1 Core Topics

Topic	Must-know for exam
Multi-agent systems	Single responsibility, shared schema, autonomy
Task decomposition	Top-down, dynamic vs plan-and-execute
Hub-and-spoke	Centralized control, spoke agents as tools
Agentic loops	ReAct pattern, termination conditions
Subagent coordination	Sequential, parallel, shared memory risks
Reliability	Idempotency, circuit breaker, checkpointing

Key Takeaways (Exam Cram)

1. **Hub-and-spoke** is the primary orchestration model for CCA-F.
2. **Task decomposition** always precedes agent assignment.
3. **Agentic loops** must have **max iterations** and **timeout**.
4. **Reliability** is not optional – always include idempotency and retries.
5. **Subagent coordination** failures = most common exam scenario for “what went wrong?”.
6. Memorize the **ReAct** flow: *Observe* → *Think* → *Act* → *Observe*.

Domain 2: Tool Design and MCP Integration (18%)

2.1. Model Context Protocol (MCP) Fundamentals

Definition

The **Model Context Protocol (MCP)** is an open standard that standardizes how AI applications (like Claude) connect with external data sources and tools. It uses a client-server architecture where MCP servers expose tools, resources, and prompts to MCP clients.

Core Architecture: Three Roles

Role	Description	Example
Host	AI application with MCP client capabilities	Claude Desktop, Claude Code, Cursor
Client	Maintains connection to one MCP server	MCP client within the host

Role	Description	Example
Server	Exposes tools, resources, and prompts	Filesystem MCP, GitHub MCP, Database MCP

Rule: One client connects to one server. Each request must be authenticated and authorized .

MCP Server Components

MCP Server

- ├ Tools → Actions Claude can perform (e.g., create_issue, read_file)
- ├ Resources → Data exposed to Claude (e.g., file contents, DB records)
- └ Prompts → Reusable prompt templates

Configuration File (.mcp.json)

Servers are registered in configuration files that Claude Desktop or VS Code reads :

```
json
{
  "inputs": [
    {
      "type": "promptString",
      "id": "github-token",
      "description": "GitHub Personal Access Token",
      "password": true
    }
  ],
  "servers": {
    "github-mcp": {
      "type": "http",
      "url": "https://mcp.github.com/api",
      "headers": {
        "Authorization": "Bearer {{inputs.github-token}}"
      }
    }
  }
}
```


Exam Tip

MCP is not just for tools—it standardizes **tools, resources, AND prompts** under one protocol. The `.mcp.json` file is the primary configuration artifact.

2.2. Designing MCP Servers and Clients

MCP Server Design Patterns

Pattern A: Logic-Throws, Handler-Catches

```
logic.ts           → Pure business logic + Zod schemas + throws on error
registration.ts    → Wraps logic, catches errors, formats responses
```

- **Why:** Keeps core logic testable; registration layer handles protocol-specific formatting
- **Exam Relevance:** High—this is the canonical pattern for production MCP servers

Pattern B: Transport-Agnostic Core

- Server logic should be independent of transport (stdio vs HTTP)
- Transports plug into the same core server

We have 700 Practice test questions **Claude Certified Architect Foundations (CCA-F)** (Taken from previous exams)

Full Practice Set link below

<https://skillcertpro.com/product/claude-certified-architect-foundations-cca-f-exam-questions/>

100% Money back Guarantee, If you don't pass the exam in 1st attempt, your money will be refunded back

MCP Client Responsibilities

Responsibility	Description
Discovery	Find available tools/resources via <code>list_tools</code>
Validation	Validate tool arguments against schema before execution
Authentication	Obtain and refresh tokens
Error handling	Parse and respond to structured errors

Practical Commands (Exam Reference)

```
# Run MCP server via stdio (default)
npm run start:server

# Run MCP server via HTTP
npm run start:server:http

# Test MCP server with inspector tool
npx @modelcontextprotocol/inspector
```

Key Takeaways

- MCP servers should be **transport-agnostic** (stdio OR HTTP)
- Use the "Logic Throws, Handler Catches" pattern for clean separation
- Clients discover capabilities dynamically—don't hardcode tool lists

2.3. Managing Tool Boundaries

Definition

Tool boundaries define the separation between:

- What the AI **proposes** (tool call arguments)
- What the system **executes** (actual API call)
- What the AI **observes** (tool response)

Boundary Design Principles

Principle	Description	Exam Scenario
Single-purpose tools	One tool = one action	create_issue, not manage_repo
Idempotency	Same input → same result	Critical for retries
Timeouts	Each tool call has max duration	Prevent hanging
Size limits	Limit input/output size	Prevent context overflow
Read-only vs Write	Separate tools for read vs write	Security boundary

Common Boundary Violations (Exam Traps)

1. **Tool does too much** → Breaks single responsibility, hard to test

2. **No input validation** → Malformed args cause runtime failures
3. **Missing error categories** → Claude can't decide to retry or give up
4. **Large responses** → Overwhelm context window

Tool Validation Pattern

```
// Pre-execution validation (safe, read-only)
validate_tool_args(tool_name, args) → {
  valid: boolean,
  errors: [{ path: ".branch", message: "required" }],
  normalized_args: object
}
```

- **Why:** Claude can dry-run validation before calling write tools
- **Exam Tip:** Look for questions about reducing live failures from bad arguments

Key Takeaways

- Tools are **boundaries**: validate inputs, limit outputs, enforce timeouts
- Separate **read** and **write** tools for security
- Pre-validation reduces production failures

2.4. SSE Transport Trade-offs

Transport Types in MCP

Transport	Direction	State	Use Case
stdio	Bidirectional (via process)	Stateful	Local, CLI tools, development
HTTP + SSE	Half-duplex: POST (client→server), SSE (server→client)	Stateless or session	Web clients, serverless
WebSocket (proposed)	Full-duplex	Stateful	Real-time, low-latency

SSE + POST Deep Dive

text

Client --(POST request)--> Server

Client <--(SSE stream)--- Server (events, progress, results)

Advantages:

-  Works with standard HTTP semantics (headers, auth, proxies)
-  Browser native (EventSource API)
-  Stateless-friendly for serverless/scale-out
-  Built-in auto-reconnect
-  Easy to debug with curl

Disadvantages:

-  Half-duplex (separate channels for send/receive)
-  More complex for bidirectional real-time

SSE vs WebSocket Comparison Table

Feature	WebSocket	HTTP + SSE
Bi-directional	 Full-duplex	 Half-duplex (POST + SSE)
HTTP headers (Auth)	 Limited	 Fully supported
Proxy/firewall	 Sometimes blocked	 Treated as normal HTTP
Debugging	 Special tools	 curl works
Stateless scaling	 Stateful connection	 Stateless POST + stream
Auto-reconnect	 Manual	 Built-in (EventSource)

When to Choose Each (Exam Decision Guide)**Choose stdio when:**

- Local CLI tools
- Development environments
- Single-process execution

Choose HTTP + SSE when:

- Web clients (browsers)
- Serverless / Lambda deployment
- Need HTTP-native auth (OAuth, headers)
- Multi-region scaling

Avoid SSE when:

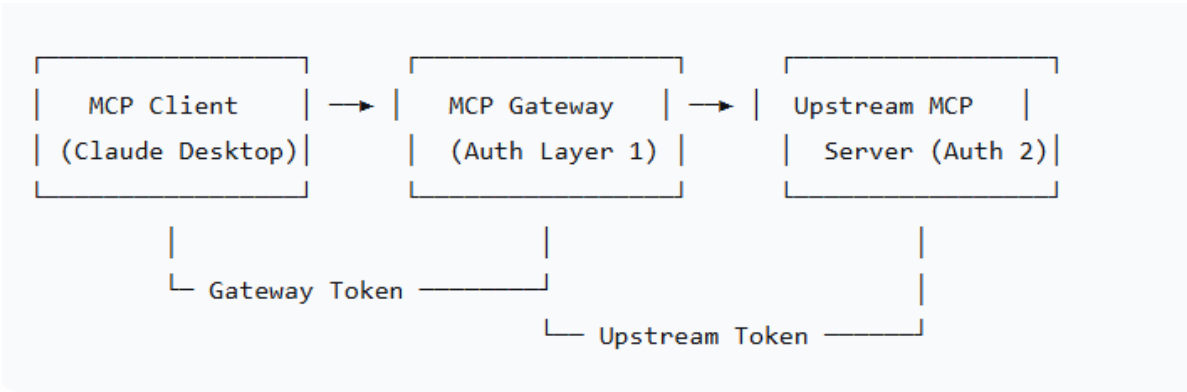
- True full duplex required (consider WebSocket)
- Very low latency needed

Key Takeaways

- MCP 2025 specification moved toward **Streamable HTTP** over pure SSE
- SSE is **stateless-friendly** (critical for exam questions about scaling)
- **Auth headers work fully with HTTP+SSE**, unlike WebSockets in browsers

2.5. Authentication Patterns for MCP

Two-Layer Authentication Model



Authentication Methods for MCP

Method	Use Case	Production Ready?	Exam Importance
API Key / Bearer Token	Local dev, quick prototypes	❌ No (insecure)	High (know the risks)
OAuth 2.1 + PKCE	Web/mobile apps, user auth	✅ Yes	★ Highest
Client Credentials	Machine-to-machine (no user)	✅ Yes	High
Identity Assertion (ID-JAG)	Enterprise SSO	✅ Yes	Medium
JWKS / OIDC	Direct IdP integration	✅ Yes	Medium

Why API Keys Are Not Production-Ready

Problem	Impact
No scope restrictions	Key grants ALL permissions
Difficult to rotate	Breaks all clients simultaneously
No expiration	Permanent risk
No user binding	No audit trail per person

Exam Stat: 53% of MCP servers rely on long-lived static credentials—this is a FAIL in production .

OAuth 2.1 + PKCE Flow (MCP Standard)

text

1. Client requests resource without token
2. Server returns 401 + WWW-Authenticate with metadata URL
3. Client fetches /.well-known/oauth-protected-resource
4. Client fetches /.well-known/oauth-authorization-server
5. Client generates PKCE code_verifier + code_challenge
6. User authorizes in browser
7. Client exchanges code + code_verifier for access token
8. Client calls MCP with Bearer token

Critical OAuth Concepts for Exam

Concept	Definition	MCP Relevance
PKCE	Proof Key for Code Exchange	Required for public clients (desktop apps)
Scope	Permission boundary	mcp scope, plus app-specific scopes
Token expiry	Limited lifetime	Refresh tokens for long sessions
Metadata discovery	/.well-known/oauth-*	Auto-configuration, no hardcoding

JWT Validation Steps (MCP Server)

1. **Signature** → Verify with IdP's JWKS public key
2. **Expiration** → Check exp and nbf claims

3. **Issuer** → Confirm iss matches expected IdP URL
4. **Audience** → Verify aud matches this MCP server's resource ID
5. **Scope** → Validate required scopes are present

Key Takeaways for Exam

- **OAuth 2.1 with PKCE** is the standard for production MCP
- **Two-layer auth**: client authenticates to gateway, gateway to upstream
- **Metadata discovery** eliminates hardcoded endpoints
- API keys are **not production-ready** (no scope, no expiry, no audit)

We have 700 Practice test questions **Claude Certified Architect Foundations (CCA-F)** (Taken from previous exams)

Full Practice Set link below

<https://skillcertpro.com/product/claude-certified-architect-foundations-cca-f-exam-questions/>

100% Money back Guarantee, If you don't pass the exam in 1st attempt, your money will be refunded back

2.6. Writing Reliable Tool Schemas

Tool Schema Components

Every MCP tool requires a JSON Schema defining its interface:

```
json

{
  "name": "create_github_issue",
  "description": "Creates a new issue in a GitHub repository",
  "inputSchema": {
    "type": "object",
    "properties": {
      "owner": { "type": "string", "description": "Repository owner" },
      "repo": { "type": "string", "description": "Repository name" },
      "title": { "type": "string", "description": "Issue title" },
      "body": { "type": "string", "description": "Issue body" },
      "labels": { "type": "array", "items": { "type": "string" } }
    },
    "required": ["owner", "repo", "title"]
  }
}
```

Schema Best Practices

Practice	Why	Example
Descriptive names	Claude selects tools by name	create_issue, not cmd1
Detailed descriptions	Helps Claude choose correctly	"Use this to create GitHub issues"
Mark required fields	Prevents missing arg errors	"required": ["owner", "repo"]
Use enums	Constrain valid values	"enum": ["bug", "feature"]
Provide examples	Clarify format	"example": "2024-01-15"
Set maxLength	Prevent overflow	"maxLength": 1000

Error Response Structure (Critical for Exam)

MCP tools must return structured errors so Claude knows whether to retry:

```

json
{
  "isError": true,
  "isRetryable": true,
  "errorCategory": "rate_limit",
  "message": "API rate limit exceeded. Retry after 60 seconds.",
  "retryAfter": 60
}

```

Error Categories for Exam:

Category	Retryable?	Example
rate_limit	✔ Yes (after delay)	Too many requests
timeout	✔ Yes	Network timeout
invalid_input	✘ No	Wrong parameters
unauthorized	✘ No	Expired token
not_found	✘ No	Resource missing
internal_error	⚠ Maybe	Server bug

Tool Description Anti-Patterns (Exam Traps)

Bad Pattern	Why It Fails
Vague description	"Updates stuff" → Claude doesn't know when to use
Missing required fields	Runtime error when arg missing
No error categories	Claude can't decide to retry or abort
Giant response	Overwhelms context window
Side effects without warning	User surprised by database write

Exam Tip

Tool descriptions drive Claude's tool selection. A tool with a poor description will never be called correctly. This is a common exam trap.

Key Takeaways

- Schemas must be **complete** (types, required, descriptions, examples)
- Errors must be **categorized** (retryable vs not)
- Tool **descriptions** are as important as implementation
- Test tool validation separately from execution

Summary Table – Domain 2 Core Topics

Topic	Must-Know for Exam
MCP Architecture	Host, Client, Server roles; .mcp.json config
MCP Server Design	Logic-Throws-Handler-Catches pattern; transport-agnostic
Tool Boundaries	Single-purpose; validate inputs; separate read/write
SSE Transport	Half-duplex; stateless-friendly; HTTP-native auth
OAuth 2.1 + PKCE	Production standard; metadata discovery; JWT validation
Tool Schemas	Complete JSON Schema; structured errors; descriptive names

Key Takeaways (Exam Cram)

1. MCP standardizes tools, resources, AND prompts—not just tools.

2. **Two auth layers:** Gateway token (client→gateway) + Upstream token (gateway→server).
3. **OAuth 2.1 + PKCE** is the production standard—API keys are for development only.
4. **SSE transport** is stateless-friendly and works with HTTP-native auth (exam loves this trade-off).
5. **Tool descriptions are critical**—bad description = tool never called.
6. **Error responses must include** `isRetryable` so Claude knows whether to retry.
7. **Logic-Throws, Handler-Catches** is the canonical MCP server pattern.

Domain 3 – Claude Code Configuration and Workflows (20%)

3.1. CLAUDE.md Hierarchy and Configuration

Definition

CLAUDE.md is a Markdown configuration file that provides persistent context, instructions, and rules to Claude Code. It is the foundation of "context engineering" – the discipline of crafting the right context for AI tools . Claude reads these files automatically on every run.

File Hierarchy and Precedence

Location	Scope	Use Case	Priority
<code>~/.claude/CLAUDE.md</code>	User-level, all projects	Personal preferences (commit style, testing approach, coding conventions)	Lowest (global defaults)
<code>./CLAUDE.md (project root)</code>	Team-shared, current project	Build commands, architecture decisions, repo etiquette, code style	Highest (primary file)
<code>./parent/directory/CLAUDE.md</code>	Monorepo package-level	Package-specific rules (auto-loaded when working in that directory)	Medium
<code>./.claude/CLAUDE.md</code>	Alternative project location	Same as root, alternative placement	Same as root

Exam Tip: The root `./CLAUDE.md` is checked into git. User-level `~/.claude/CLAUDE.md` is private to the developer.

What to Include vs. Exclude

✓ Include	✗ Exclude
Bash commands Claude can't guess	Anything Claude can figure out by reading code
Code style rules that differ from defaults	Standard language conventions Claude already knows
Testing instructions & preferred runners	Detailed API docs (link instead using @docs/)
Repo etiquette (branch naming, PR rules)	Information that changes frequently
Architecture decisions & environment quirks	File-by-file codebase descriptions
Common gotchas & non-obvious behaviors	Self-evident practices like "write clean code"

Effective CLAUDE.md Template (Next.js/TypeScript)

markdown

Commands

- Dev: ``bun run dev`` (port 3000)
- Test: ``bun test`` (single: ``bun test path/to/file``)
- Lint: ``bun run lint``
- Typecheck: ``bun run typecheck``

Code style

- Use ES modules (import/export), not CommonJS
- Use TypeScript strict mode – no ``any`` types

Architecture

- Next.js 15 App Router, React 19
- Database: PostgreSQL with Drizzle ORM
- See @docs/api-patterns.md for REST conventions

Workflow

- IMPORTANT: Always typecheck after a series of code changes
- Never push directly to main – create a branch and PR

Gotchas

- IMPORTANT: Never import from @/lib/server in client components

Emphasis Markers

Adding IMPORTANT or YOU MUST on critical rules measurably improves Claude's compliance. Use sparingly – if everything is important, nothing is .

.claudeignore – Context Sanitization

Works like .gitignore. Prevents Claude from wasting context on irrelevant files.

```
gitignore
```

```
# Dependencies
```

```
node_modules/
```

```
.venv/
```

```
# Build artifacts
```

```
dist/
```

```
build/
```

```
.next/
```

```
# Generated files
```

```
*.lock
```

```
*.min.js
```

```
coverage/
```

```
# Large assets
```

```
public/assets/*.png
```

Key Takeaways

- CLAUDE.md is **mandatory** for production use – not optional
- Root CLAUDE.md is **checked into git** for team sharing
- User-level ~/.claude/CLAUDE.md is for **personal preferences**
- .claudeignore reduces token waste significantly
- Use IMPORTANT sparingly for critical rules only

3.2. Custom Slash Commands

Definition

Slash commands are frequently-used prompts defined as Markdown files that Claude executes during interactive sessions. They enable reusable, consistent workflows that can be shared across teams .

Command Locations and Priority

Type	Location	Scope	Priority
Project	.claude/commands/	Current project only (shared via git)	Highest
Personal	~/claude/commands/	All projects (private)	Medium
Plugin	[plugin-name]/commands/	Plugin-specific	Lowest

Exam Tip: Project-level commands override personal commands with the same name. This enables team standardization.

File Structure

Basic command (no frontmatter required):

```
.claude/commands/
├─ review.md      # /review command
├─ test.md        # /test command
└─ deploy.md      # /deploy command
```

YAML Frontmatter Fields

Field	Type	Purpose	Example
description	String	Brief description shown in /help	"Review PR for security issues"
allowed-tools	String/Array	Specify which tools the command can use	Read, Grep, Bash(git:*)
model	String	Model for execution (sonnet, opus, haiku)	haiku (fast) vs opus (complex)
argument-hint	String	Document expected arguments for autocomplete	[pr-number] [priority]
disable-model-invocation	Boolean	Prevent programmatic invocation via SlashCommand tool	true

Command with Frontmatter Example

```
markdown

---
description: Review code for security vulnerabilities
allowed-tools: Read, Grep, Bash(git:*)
model: sonnet
argument-hint: [file-path]
---

Review this code for:
- SQL injection
- XSS attacks
- Authentication bypass
```

We have 700 Practice test questions **Claude Certified Architect Foundations (CCA-F)** (Taken from previous exams)

Full Practice Set link below

<https://skillcertpro.com/product/claude-certified-architect-foundations-cca-f-exam-questions/>

100% Money back Guarantee, If you don't pass the exam in 1st attempt, your money will be refunded back

Dynamic Features

Arguments:

Type	Syntax	Example
All arguments	\$ARGUMENTS	/fix 123 → "Fix issue #123"
Positional	\$1, \$2, \$3	/review 456 high → \$1="456", \$2="high"

Bash Execution (requires `allowed-tools: Bash(...)`):

markdown

`allowed-tools: Bash(git status:*), Bash(git diff:*)`

Current status: `!`git status``

Changes: `!`git diff HEAD``

File References:

markdown

Review `@src/utils/helpers.js` for issues.

Compare `@$1` with `@$2`.

Namespacing via Subdirectories

- `.claude/commands/frontend/component.md` → `/component` (shown as "project:frontend")
- `~/claude/commands/backend/api.md` → `/api` (shown as "user:backend")

SlashCommand Tool

The SlashCommand tool allows Claude to programmatically invoke custom commands. Has a default character budget of **15,000 characters** to prevent token overflow. Set `disable-model-invocation: true` to prevent automatic triggering .

Key Takeaways

- Commands are **Markdown files** – no special binary format
- **Frontmatter** is optional but recommended for production commands
- `$ARGUMENTS` captures everything; `$1` captures first word
- `!command` executes bash; `@file` includes file content
- Project commands **override** personal commands with same name

3.3. CI/CD Pipeline Integration

Definition

Integrating Claude Code into CI/CD pipelines enables automated PR reviews, issue triage, code generation, and maintenance tasks. The official anthropics/claude-code-action is the primary method for GitHub Actions.

Execution Modes

Mode	Trigger	Behavior
Interactive	@claude mention in PR/issue comment	Reads context, responds, can push commits
Automation	Workflow with prompt parameter	Runs prompt immediately, outputs results
Review	PR opened/synchronized event	Analyzes PR diff, posts review comments

Minimal GitHub Actions Workflow

```
name: Claude Code PR Review
on:
  pull_request:
    types: [opened, synchronize]

jobs:
  review:
    runs-on: ubuntu-latest
    permissions:
      contents: read
      pull-requests: write
    steps:
      - uses: actions/checkout@v4
      - uses: anthropics/claude-code-action@v1
        with:
          anthropic_api_key: ${ secrets.ANTHROPIC_API_KEY }
          prompt: "/review"
          claude_args: "--max-turns 5"
```


@claude Comment Trigger

```

yaml
  issue_comment:
    types: [created]

  jobs:
    respond:
      if: contains(github.event.comment.body, '@claude')
      runs-on: ubuntu-latest
      permissions:
        contents: read
        pull-requests: write
        issues: write
      steps:
        - uses: actions/checkout@v4
        - uses: anthropics/claude-code-action@v1
          with:
            anthropic_api_key: ${ secrets.ANTHROPIC_API_KEY }
            # No prompt → interactive mode

```

Authentication Options

Provider	Secret Required	Best For
Anthropic Direct	ANTHROPIC_API_KEY	Most teams, fastest setup
AWS Bedrock	AWS_ROLE_TO_ASSUME (OIDC)	AWS-native orgs, data residency
Google Vertex AI	GCP_WORKLOAD_IDENTITY_PROVIDER	GCP-native orgs

We have 700 Practice test questions **Claude Certified Architect Foundations (CCA-F)** (Taken from previous exams)

Full Practice Set link below

<https://skillcertpro.com/product/claude-certified-architect-foundations-cca-f-exam-questions/>

100% Money back Guarantee, If you don't pass the exam in 1st attempt, your money will be refunded back

AWS Bedrock Example

```
yaml

- name: Configure AWS Credentials (OIDC)
  uses: aws-actions/configure-aws-credentials@v4
  with:
    role-to-assume: ${ secrets.AWS_ROLE_TO_ASSUME }
    aws-region: us-west-2

- uses: anthropics/claude-code-action@v1
  with:
    use_bedrock: "true"
    claude_args: '--model us.anthropic.claude-sonnet-4-6'
```

Configuration Reference

Parameter	Required	Description
anthropic_api_key	Yes (for direct)	Anthropic API key
prompt	No	Instructions for Claude. Omit for interactive mode.
claude_args	No	CLI arguments: --max-turns, --model, --mcp-config
trigger_phrase	No	Custom trigger phrase (default: @claude)
use_bedrock	No	Set 'true' for AWS Bedrock

Common claude_args

```
yaml

claude_args: |
  --max-turns 10           # Limit conversation turns (default: 10)
  --model claude-opus-4-6 # Use Opus instead of Sonnet
  --mcp-config /path/to/config.json
  --allowed-tools "Read,Write,Bash"
  --append-system-prompt "Follow our coding standards"
```

Path-Specific Triggers

```
yaml

on:
  pull_request:
    paths:
      - 'src/auth/**'      # Only trigger for auth changes
      - 'src/api/**'
```

Scheduled Maintenance

```
yaml

on:
  schedule:
    - cron: '0 9 * * 1'    # Every Monday at 9am UTC
```

Cost Control Best Practices

Control	Purpose
<code>timeout-minutes: 10</code>	Kill runaway jobs
<code>concurrency: cancel-in-progress: true</code>	Cancel if new push arrives
<code>--max-turns 5</code>	Cap iterations for reviews
<code>--max-turns 10-15</code>	Implementation tasks

Token Usage Estimates

Task Type	Typical Token Usage
Simple PR review	5,000 – 15,000 tokens
Feature implementation	50,000 – 200,000 tokens

Portability: GitLab CI / Jenkins

For non-GitHub environments, either:

1. **Script the CLI directly** (claude command)
2. **Call Anthropic API** directly via curl

Caveat: Some environments report /login requirement in headless CI. Validate before automating.

Security Hardening

- Pin action to **commit SHA**, not just version tag
- Use **least-privilege permissions** (never contents: write unless needed)
- Store ANTHROPIC_API_KEY as **repository secret**
- Never echo secrets in logs

Key Takeaways

- anthropics/claude-code-action@v1 is the official GitHub Action
- Three modes: **interactive** (@claude), **automation** (prompt), **review** (PR events)
- **OIDC** for Bedrock/Vertex – no static credentials
- Always set --max-turns for cost control
- Pin actions by **commit SHA** for supply chain security

3.4. Persistent Project Context

The Problem

- **Cross-session memory loss:** Session 1 builds auth system → Session 2 has no idea it exists
- **Auto-compact:** At ~200K tokens, context is compressed → ~90% lost
- No external tool can prevent this natively – Claude Code loads everything into system prompt

Solutions Overview

Tool	Purpose	Method
claude-memory-hub	Persistent memory across sessions	PreCompact/PostCompact hooks, SQLite storage
memory-bank	Structured project knowledge	Decisions, patterns, architecture docs
Project Knowledge (methodology)	Just-in-time context loading	Separate skill files, load only what's needed

claude-memory-hub – Compact Interceptor (The Innovation)

How it works:

BEFORE compact → PreCompact hook scores entities → Injects priority list → Compact engine knows what to preserve
AFTER compact → PostCompact hook captures full summary → Saves to SQLite L3 → Zero information loss

Key Features:

- **Compact Interceptor:** Prevents 90% context loss during auto-compact
- **Cross-session memory:** Past context auto-injected based on current work
- **Hybrid search:** FTS5 (keyword) + TF-IDF + semantic embeddings (384-dim)
- **3-layer privacy:** <private> tags, auto secret detection, path filtering
- **Slash commands:** /mem-search, /mem-status, /mem-save

Installation:

```
bunx claude-memory-hub install    # Registers MCP server + 5 hooks + 3 commands
bunx claude-memory-hub status    # Verify
```

Memory Bank System

Core Components:

Directory	Purpose
decisions/	Architecture Decision Records (ADRs)
patterns/	Code patterns and conventions
architecture/	System structure and components
troubleshooting/	Known issues and solutions

Key Commands:

Command	Purpose
/context-query "<pattern>" --limit=15	Get focused context for a feature/path
/update-memory-bank	Sync memory bank with current code
/context-diff --since=main	See what changed
/cleanup-context	Archive current state for milestones

Installation:

```
bash
```

```
git clone git@github.com:russebeye/claude-memory-bank.git
```

```
cp -r {agents,commands,workflows} ~/.claude/
```

```
cp CLAUDE.md ~/.claude/
```

Project Knowledge Methodology

Files stored in `~/.claude/skills/project-knowledge/references/`:

File	Content
project.md	Purpose, audience, key features, scope
architecture.md	Tech stack, structure, dependencies, data model
patterns.md	Code conventions, git workflow, testing strategy
deployment.md	Platform, env vars, CI/CD, monitoring
ux-guidelines.md	UI language, tone, domain glossary

Key principle: CLAUDE.md stays minimal. Agent loads only needed files (just-in-time context), not all context.

Token Economics Comparison

Metric	Value
Claude Code vs. Cursor	5.5x fewer tokens
Context saved with subagent search	40%+
Auto-compact threshold	70% (not 90%)

Key Takeaways

- **Native Claude Code has NO persistent memory** – use extensions
- claude-memory-hub solves the auto-compact problem uniquely (PreCompact/PostCompact hooks)
- **Memory Bank** provides structured, queryable project knowledge
- **Project Knowledge** follows just-in-time loading (not all context)
- **Rule:** Start fresh sessions per task – long sessions degrade quality

3.5. AI-First Development Workflows

Definition

AI-first development means designing workflows where AI agents are the primary executors, with humans in review/approval roles. The methodology emphasizes **specification before implementation** and **automated validation**.

The Butterfly Effect Prevention

The model misunderstands something early → builds entire feature on faulty premises → nobody notices until 5 PRs deep.

Fix: Ruthlessly tight task scoping.

Vague Prompt (Causes Butterfly Effect)	Scoped Prompt (Prevents It)
"add tests for foo.py"	"write a test for foo.py covering edge case where user is logged out. avoid mocks."
"fix the login bug"	"login fails after session timeout. check auth flow in src/auth/, especially token refresh. write a failing test, then fix it."

The Interview Pattern

For larger features, have Claude interview you before building:

markdown

I want to build [brief description]. Interview me in detail using the AskUserQuestion tool.

Ask about technical implementation, UI/UX, edge cases, concerns, and tradeoffs.

Keep interviewing until covered, then write a complete spec to SPEC.md.

Once spec is complete, start a fresh session to execute it.

Plan Mode Workflow

Phase	Mode	Action
Explore	Plan Mode	Claude reads files, answers questions without changes
Plan	Plan Mode	Create detailed implementation plan
Implement	Normal Mode	Claude codes against plan, verifying as it goes
Commit	Normal Mode	Commit with descriptive message, open PR

Toggle Plan Mode: Shift+Tab (twice)

When to skip Plan Mode: Fixing a typo, adding a log line, renaming a variable – if you can describe diff in one sentence, skip planning.

Feature Development Pipeline (Spec-Driven)

Step 1: /new-user-spec → Interview → User spec (Russian, human-readable)

Step 2: /new-tech-spec → Research → Tech spec (English, for agents)

Step 3: /decompose-tech-spec → Task breakdown → Individual task files

Step 4: /do-feature OR /do-task → Implementation + QA

Step 5: /done → Archive + update documentation

Specialized Agents in Pipeline:

Agent	Purpose
interview-completeness-checker	Ensure no gaps in user spec
userspec-quality-validator	Structure, testability of acceptance criteria
skeptic	Find mirages (nonexistent files/functions/APIs)
security-auditor	OWASP Top 10 checks
code-searcher	Deep codebase analysis

Context Hygiene Rules

Command	When to Use	Effect
/clear	Between unrelated tasks	Resets entire context window
/compact	When context is 70%+ full	Summarizes while preserving key info
Esc + Esc	Mid-session recovery	Rewind to checkpoint
Subagents	Codebase research	Searches in separate context, returns summary

The Two-Correction Rule

If you've corrected Claude more than twice on the same issue, run /clear and start fresh. A clean session with a better prompt outperforms a long session with accumulated corrections.

What to Automate vs. Keep Manual

 Automate	 Keep Manual
PR review hints (risk deltas, perf issues)	High-risk refactors without tests
Changelog & docs smoothing	Big-bang rewrites (require human merge)
Issue triage → repro steps	Security-sensitive changes

Workflow Commands Summary

Command	Purpose
<code>/init</code>	Generate starter CLAUDE.md
<code>/clear</code>	Reset context between tasks
<code>/compact [focus]</code>	Summarize context, optionally with focus
<code>/review</code>	Code review (often in CI)
<code>/update-memory-bank</code>	Sync persistent memory
<code>/context-query</code>	Retrieve relevant past context

Key Takeaways

- **Spec before code:** User spec → Tech spec → Tasks → Implementation
- **Plan Mode** separates research from implementation (prevents wrong solutions)
- **Two-correction rule:** After 2 corrections, `/clear` and restart
- **The Interview Pattern:** Have Claude interview you for complex features
- **Batch reviews** – don't approve every single edit individually
- **Subagents** for codebase research (returns summary, not raw context)

Summary Table – Domain 3 Core Topics

Topic	Must-Know for Exam
CLAUDE.md Hierarchy	Root (git) > User (~/.claude) > Parent (monorepo)
CLAUDE.md Content	Include: commands, style, architecture, gotchas. Exclude: obvious defaults

Slash Commands	.md files in .claude/commands/; frontmatter optional; \$ARGUMENTS, !command, @file
CI/CD Integration	anthropics/claude-code-action@v1; 3 modes: interactive, automation, review
Authentication	Direct (API key) vs. Bedrock (OIDC) vs. Vertex (OIDC)
Cost Control	--max-turns, timeout-minutes, concurrency limits
Persistent Context	claude-memory-hub (compact interceptor); Memory Bank (structured docs)
AI-First Workflow	User spec → Tech spec → Tasks → Implementation → Done
Plan Mode	Shift+Tab twice; explore → plan → implement

Key Takeaways (Exam Cram)

1. **CLAUDE.md is mandatory** – treat it like .gitignore. Root version checked into git.
2. **Slash commands** are Markdown files – use frontmatter for allowed-tools, model, argument-hint.
3. **CI/CD uses** anthropics/claude-code-action@v1 – three modes: @claude mention, prompt: parameter, or PR event.
4. **Cost control** = --max-turns – 5 for reviews, 10-15 for implementation.
5. **Persistent memory is NOT native** – use claude-memory-hub or Memory Bank extension.
6. **PreCompact/PostCompact hooks** are the innovation – they salvage context during auto-compact.
7. **Two-correction rule** – after 2 corrections, /clear and restart fresh.
8. **Plan Mode** (Shift+Tab **twice**) separates thinking from doing – prevents butterfly effect.

Domain 4 – Prompt Engineering and Structured Output

4.1. JSON Schema Enforcement

Definition

JSON Schema enforcement is the practice of guaranteeing that Claude's output matches a predefined JSON structure. This can be achieved through native constrained decoding (--json-schema flag) or through prompt-based techniques with validation retries.

Native Constrained Decoding (CLI)

The --json-schema flag provides guaranteed schema compliance at the token generation level—Claude physically cannot produce tokens that violate the schema.

Command Syntax:

```
bash
```

```
claude -p "List 3 countries with capitals" \
  --output-format json \
  --json-schema countries.schema.json
```

Schema File Example (countries.schema.json):

```
json
{
  "type": "array",
  "items": {
    "type": "object",
    "required": ["country", "capital"],
    "properties": {
      "country": {"type": "string"},
      "capital": {"type": "string"}
    }
  }
}
```

Guaranteed Output:

```
json
[
  {"country": "France", "capital": "Paris"},
  {"country": "Germany", "capital": "Berlin"},
  {"country": "Japan", "capital": "Tokyo"}
]
```

Without Native Enforcement (Prompt-Based)

When --json-schema is unavailable, use prompt-based guidance with validation retries:

```
python
RESULT_SCHEMA = {
    "type": "object",
    "required": ["ok"],
    "properties": {
        "ok": {"type": "boolean"},
        "summary": {"type": "string"}
    },
    "additionalProperties": False
}
```

```
def call_with_schema(prompt: str, schema: dict, client) -> dict:
    response = client.messages.create(
        model="claude-sonnet-4-6",
        messages=[{
            "role": "user",
            "content": f""Return ONLY JSON matching this schema: {schema}
            Task: {prompt}""
        }]
    )
    return json.loads(response.content[0].text)
```

Schema Design Best Practices

Practice	Why	Example
"additionalProperties": false	Prevents extra fields	Required for strict validation
Use "required" array	Enforces mandatory fields	["id", "name"]
Set "type" on all properties	Type safety	"type": "number"
Use "enum" for constrained values	Limits possible outputs	["positive", "negative"]

Exam Tip

The --json-schema flag is the **preferred method** for structured output because it provides **100% guaranteed compliance** via constrained decoding. Prompt-based approaches require validation retry loops.

We have 700 Practice test questions **Claude Certified Architect Foundations (CCA-F)** (Taken from previous exams)

Full Practice Set link below

<https://skillcertpro.com/product/claude-certified-architect-foundations-cca-f-exam-questions/>

100% Money back Guarantee, If you don't pass the exam in 1st attempt, your money will be refunded back

4.2. Few-Shot Techniques

Definition

Few-shot prompting provides example input-output pairs in the prompt to teach Claude the desired format and behavior. This is more reliable than instruction alone for structured outputs.

When to Use Few-Shot

Scenario	Few-Shot Effectiveness
Unusual output format	✓ High (demonstrates exact format)
Complex classification	✓ High (shows edge cases)
Standard JSON	✗ Low (instruction sufficient)
Common task (summarization)	✗ Low (model already knows)

How Many Examples?

Rule of thumb: 2–5 examples. Beyond 5, additional examples rarely improve quality and waste tokens.

Example Count	Effectiveness	Token Cost
0 (zero-shot)	Variable	Lowest
1-2	Good	Low
3-5	Optimal	Medium
6-10	Diminishing returns	High
10+	No benefit	Very High

Few-Shot Prompt Template

python

```
async def get_few_shot_prompt(query: str) -> str:
    examples = [
        {"input": "How do I reset my password?",
         "output": "Go to Settings > Security > Reset Password"},
        {"input": "Where can I see my order history?",
         "output": "Navigate to Account > Orders"},
        {"input": "How do I contact support?",
```

```

        "output": "Click Help > Contact Us or email support@example.com"},
    ]

    examples_text = "\n".join(
        f"User: {ex['input']}\nAssistant: {ex['output']}"
        for ex in examples[:3] # Use 3 examples maximum
    )

    return f"""You are a helpful customer support assistant.

```

Here are some example interactions:

```
{examples_text}
```

Now respond to this query:

```
User: {query}
```

```
Assistant: ""
```

Dynamic Example Selection

For production systems, select examples based on semantic similarity to the current query:

```
python
```

```

from langchain_core.example_selectors import SemanticSimilarityExampleSelector

example_selector = SemanticSimilarityExampleSelector.from_examples(
    examples=example_pool, # Your full example database
    embeddings=embedding_model,
    k=2 # Select 2 most similar examples
)

selected_examples = await example_selector.aselect_examples({"input": query})

```

Key Takeaways

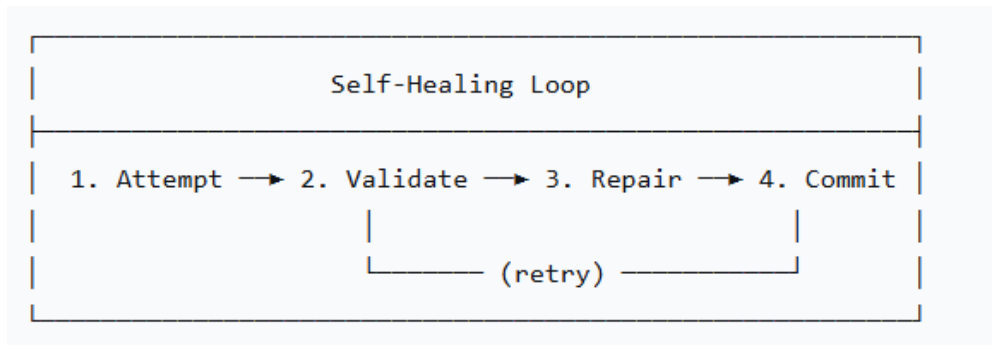
- **2-5 examples** is the optimal range for few-shot prompting
- Examples teach format through **demonstration**, not instruction
- Dynamic selection via **semantic similarity** improves relevance
- Few-shot is **not always better**—use only when instruction alone fails

4.3. Validation Retry Loops

Definition

A **validation retry loop** is a programmatic pattern that calls Claude, validates the output against a schema or deterministic tests, and retries with targeted repair prompts when validation fails.

Core Loop Architecture



Complete Python Implementation

python

```

import json, time, random
from jsonschema import Draft7Validator
import pybreaker

# Configuration
MAX_ATTEMPTS = 5
DOUBLE_PASS_REQUIRED = 2 # Two consecutive passes before commit
WALL_CLOCK_BUDGET_SEC = 60

# Schema
RESULT_SCHEMA = {
    "type": "object",
    "required": ["ok"],
    "properties": {
        "ok": {"type": "boolean"},
        "summary": {"type": "string", "maxLength": 200}
    },
    "additionalProperties": False
}

validator = Draft7Validator(RESULT_SCHEMA)

def run_validators(obj: dict) -> list:
    """Returns list of validation errors (empty = valid)."""
    errors = []
    # Schema validation
    for error in validator.iter_errors(obj):

```

```

        errors.append(f"Schema: {error.message}")
# Semantic validation
if obj.get("ok") is True:
    summary = obj.get("summary", "")
    if not summary or len(summary) > 200:
        errors.append("Test: When ok is true, provide summary <= 200 chars")
return errors

def full_jitter_delay(attempt: int, base: float = 0.5, cap: float = 10) -> float:
    """Exponential backoff with full jitter - prevents retry storms."""
    return random.uniform(0, min(cap, base * (2 ** attempt)))

# Circuit breaker
breaker = pybreaker.CircuitBreaker(fail_max=5, reset_timeout=30)

def self_heal_loop(task_instructions: str, client) -> dict:
    start = time.time()
    attempts = 0
    consecutive_passes = 0
    last_valid = None

    base_messages = [{
        "role": "user",
        "content": f"Return ONLY JSON matching schema. Task: {task_instructions}"
    }]

    while attempts < MAX_ATTEMPTS and (time.time() - start) < WALL_CLOCK_BUDGET_SEC:
        attempts += 1

        try:
            resp = breaker.call(client.messages.create, **call_params)
            text = resp.content[0].text
            obj = json.loads(text)
        except Exception as e:
            time.sleep(full_jitter_delay(attempts))
            continue

        problems = run_validators(obj)

        if not problems:
            consecutive_passes += 1
            last_valid = obj

```



```

    if consecutive_passes >= DOUBLE_PASS_REQUIRED:
        return {"status": "pass", "attempts": attempts, "result": obj}
    else:
        consecutive_passes = 0
        # Targeted repair prompt
        repair_msg = {
            "role": "user",
            "content": f""Previous output failed validation.
Problems: {'; '.join(problems)}
Return ONLY corrected JSON matching the schema.""
        }
        base_messages.append(repair_msg)
        time.sleep(full_jitter_delay(attempts))

    return {"status": "timeout", "attempts": attempts, "result": last_valid}

```

Double-Pass Requirement

The `DOUBLE_PASS_REQUIRED = 2` pattern ensures reliability by requiring **two consecutive successful validations** before committing. This prevents:

- Fluke successes from lucky generation
- Edge cases where validation missed a semantic issue

Exponential Backoff with Jitter

Why jitter matters: Without jitter, multiple failed requests retry simultaneously, causing "retry storms" that overwhelm downstream services.

Retry Pattern	Behavior	Best For
Fixed delay	Same wait every time	Simple, low-volume systems
Linear backoff	attempt * base	Moderate scale
Exponential backoff	base * (factor^attempt)	High scale
Full jitter	Random between 0 and cap	Production systems (prevents storms)

Key Takeaways

- **Double-pass validation** ensures reliability before commit
- **Exponential backoff with jitter** prevents retry storms
- **Circuit breaker** protects downstream services after repeated failures
- **Repair prompts** must be targeted (feed back specific validation errors)

We have 700 Practice test questions **Claude Certified Architect Foundations (CCA-F)** (Taken from previous exams)

Full Practice Set link below

<https://skillcertpro.com/product/claude-certified-architect-foundations-cca-f-exam-questions/>

100% Money back Guarantee, If you don't pass the exam in 1st attempt, your money will be refunded back

4.4. Programmatic Enforcement vs. Prompt-Based Guidance

The Critical Distinction

Prompts are probabilistic suggestions; enforcement is deterministic action. This distinction is the most important concept in the entire domain.

Comparison Table

Aspect	Prompt-Based Guidance	Programmatic Enforcement
Mechanism	Instructions in system prompt/CLAUDE.md	Code that intercepts and validates tool calls
Determinism	✗ Probabilistic (model may ignore)	✓ Deterministic (always applies)
Prompt Injection	✗ Vulnerable (>90% success rate)	✓ Immune
Model Updates	✗ May break behavior	✓ Unaffected
Multi-Step Safety	✗ Check only at input boundary	✓ Check before every action
Auditability	✗ Difficult to prove compliance	✓ Logged and verifiable

Why Prompt Guardrails Fail in Production

Three failure modes:

Failure Mode	Description	Real-World Impact
Prompt Injection	Attacker embeds instructions in data Claude reads	Model follows attacker's commands instead of system prompt

Failure Mode	Description	Real-World Impact
Multi-Step Reasoning	Clean input at step 1 triggers dangerous tool call at step 3	No single message appears malicious
Model Updates	Guardrail tuned for old model version	New model has different probability distribution

Example of the Gap:

python

```
# The moment between intent and execution - prompt guardrails don't live here!
tool_call = {"name": "shell/execute", "arguments": {"command": "rm -rf /"}}
# ↑ Prompt guardrails already failed if we reached this point
result = execute_shell(command) # Runtime enforcement lives here
```

Programmatic Enforcement Pattern

With runtime enforcement layer (e.g., Faramesh):

python

```
# Policy definition (enforced in code, not prompts)
agent payment-bot {
    default deny # Deny by default - explicit permission required

    rules {
        deny! shell/* reason: "never shell" # Compile-time guarantee
        defer stripe/refund
            when amount > 500
            notify: "finance"
            reason: "high value refund needs approval"
        permit stripe/* when amount <= 500
    }
}
```

Key features of enforcement:

- deny! effect provides **compile-time guarantee**—no subsequent rule can override
- Deferral requires **human approval** before execution
- Policies are **deterministic**—model cannot override

Practical Guidance: Layered Defense

Layer	Purpose	Implementation
Layer 1: Prompt Guidance	Development, non-critical tasks	CLAUDE.md, system prompts
Layer 2: Tool Validation	Production safety	Validate inputs before execution
Layer 3: Runtime Enforcement	Security-critical actions	Policy engine intercepting tool calls

Exam Scenario Identification

Question Clue	Correct Answer
"Security boundary for production agent"	Runtime enforcement layer
"Model keeps ignoring safety instruction"	Add tool-level validation
"Prompt injection from retrieved documents"	XML tags + explicit immunity instruction
"100% guarantee needed for schema compliance"	--json-schema flag or constrained decoding

Key Takeaways

- **Prompt guardrails are suggestions**, not enforcement
- The **gap lives between tool call decision and execution**—this is where enforcement must happen
- **Runtime enforcement** intercepts every tool call with deterministic policies
- **Deny-by-default** with explicit permissions is the production standard
- For the exam: If a scenario requires **guaranteed safety**, choose **programmatic enforcement** over prompt-based guidance

Summary Table – Domain 4 Core Topics

Topic	Must-Know for Exam
JSON Schema Enforcement	--json-schema flag provides guaranteed compliance; prompt-based requires retry loops
Few-Shot Techniques	2-5 examples optimal; use semantic similarity for dynamic selection
Validation Retry Loops	Double-pass validation; exponential backoff with jitter; circuit breaker

Topic	Must-Know for Exam
Programmatic vs. Prompt	Prompts are suggestions; enforcement is deterministic; gap lives between intent and execution

Key Takeaways (Exam Cram)

1. **Native** --json-schema provides guaranteed compliance—always prefer over prompt-based for production.
2. **2-5 examples** is the optimal few-shot range—more wastes tokens without benefit.
3. **Double-pass validation** ensures reliability before committing outputs.
4. **Exponential backoff with jitter** prevents retry storms in production systems.
5. **Prompt guardrails are suggestions**—they fail against injection, multi-step reasoning, and model updates.
6. **The enforcement gap** lives between tool call intent and execution—this is where runtime enforcement must act.
7. **Deny-by-default** with deny! compile-time guarantees is the production security standard.
8. For the exam: **Structured CoT with explicit reasoning scaffold** outperforms basic "think step by step" on complex tasks.

Domain 5 – Context Management and Reliability (15%)

5.1. CALM Framework (Calm Technology for Agents)

Definition

The **CALM framework** (Calm Technology for Autonomous Agents) is a design philosophy that emphasizes peripheral awareness, natural context switching, and human control. It originates from Mark Weiser's work on ubiquitous computing at Xerox PARC in the 1990s .

Core Principle

Information should transition **smoothly from the periphery to the center of attention**—not through interrupts.

The Three Legs of CALM for Autonomous Agents

For an async background agent to be truly "calm," three properties must hold simultaneously:

Property	Description	Implementation
Full Autonomy	Agent doesn't stop to ask clarifying questions mid-task	Eliminates the "stay interruptible" burden on the user

Property	Description	Implementation
Sandboxed Evolution	Agent works in isolation (worktrees, containers)	User monitors results only, not safety
Legible Progress Signals	Structured peripheral indicators (tests passing, scope narrowing)	Dashboard or status bar—not chat logs requiring focused reading

Exam Insight: Remove any one leg and the agent becomes interruptive. The analogy: delegating to a junior engineer with their own dev environment vs. pair programming with someone who taps your shoulder repeatedly.

CALM vs. Traditional Agent Design

Aspect	Traditional (Interruptive)	CALM (Peripheral)
Progress communication	Chat logs requiring focus	Dashboard counters (tests passing, scope narrowing)
Failure handling	Immediate alert	Batched at natural breakpoints
User re-engagement	Push notifications	Pull when ready (CI dashboard model)
Context switching cost	High (interrupts flow state)	Low (you choose when to check)

Application to Agentic Systems

In CCA-F context, CALM principles apply when designing:

- Background agents that run long tasks
- Delegation patterns where you fire off agents in parallel
- Systems where multiple agents return results at different times

Key Takeaway: Results should accumulate quietly and be pulled into focus at natural breakpoints you choose, not pushed via interrupts when agents finish.

5.2. Prompt Caching with `cache_control` Breakpoints

Definition

Prompt caching is an Anthropic API feature that caches stable portions of your prompt (system instructions, tool definitions, conversation history) server-side. Cache reads cost **0.1×** the normal input price—a 90% reduction.

Why Claude Code Needs Caching Aggressively

In Claude Code, the input/output token ratio can be **36:1**—each request is input-heavy:

Single "fix a bug" request token distribution:

└ system prompt (system + 24 tool definitions)	≈ 3,200 tokens
└ CLAUDE.md (project instructions)	≈ 800 tokens
└ conversation history (5 turns)	≈ 8,500 tokens
└ current file content	≈ 2,000 tokens
└ user new message	≈ 50 tokens

Input total: 14,550 tokens

Output: ≈ 400 tokens

Input/Output ratio: 36:1

Exam Point: Without caching, each input token's cost dominates the bill. Caching is not "minor optimization"—it's a quantity-level difference.

How `cache_control` Works: Prefix Matching

Critical to understand: Caching is based on **prefix matching**, not content de-duplication. The server caches your prompt prefix (from the first token). If the prefix changes anywhere, everything after that point becomes uncacheable .

Example:

```
Request 1: [Sys][Tools][CLAUDE.md v1][Conversation 1-5][New Message A]
           ↓ Cache written

Request 2: [Sys][Tools][CLAUDE.md v1][Conversation 1-5][New Message B]
           ↓ Cache HIT (prefix matches)

Request 3: [Sys][Tools][CLAUDE.md v2][Conversation 1-6][New Message C]
           ↑ CLAUDE.md changed
           → Entire prefix up to that point is now different → CACHE MISS
```

`cache_control` Breakpoint Placement

The `cache_control` marker is placed on message blocks. The optimal strategy :

Content Type	Strategy	Breakpoint Placement
System prompt	Cache on first turn, reused every turn	At system prompt end
Tool definitions	Cache once per session	At tools block end
Large user messages	Cache when exceeding threshold	At message end
CLAUDE.md	Cache separately per project	On CLAUDE.md message

Cache TTL Options

TTL	Write Price	Read Price	Break-Even Point	Best For
5 minutes	1.25× base	0.1× base	1 hit (28% of write cost)	High-frequency continuous work (<5 min between requests)
1 hour	2.0× base	0.1× base	2 hits (1.12× write cost)	Task switching, long breaks, cross-lunch sessions

Real-World Savings Benchmarks

Session Type	Turns	Without Caching	With Caching	Savings
Bug fix (single file)	20	184,000 tokens	28,400 tokens	85%
Refactor (5 files)	15	310,000 tokens	61,200 tokens	80%
General coding	40	890,000 tokens	71,200 tokens	92%

Source:

Common Cache Failure Traps (Exam Critical)

Trap	Why It Fails	Solution
Modifying CLAUDE.md mid-session	Changes prefix → invalidates all caches after that point	Plan CLAUDE.md before session; use /new to change
Dynamic timestamps in system prompt	f"Current time: {datetime.now()}" → prefix changes every request	Put dynamic content at end of messages, not in system prompt
Switching models mid-session	Different model = different cache namespace	Stay consistent; use /new if switching
/compact command	Uses different system prompt (no tools) → different prefix	Compact early before accumulating many turns
/resume or --resume	Message structure differences → perceived as new prefix	Complete tasks in single session when possible

Source:

Minimum Cache Token Thresholds

Model	Minimum Cache Size
Sonnet 4, Opus 4	~1,024 tokens
Opus 4.6, Haiku 4.5	~4,096 tokens

Exam Tip: If your cacheable block is below the threshold, `cache_control` is silently ignored—no error, just no savings.

Best Practices for High Cache Hit Rate

1. **Keep `CLAUDE.md` under 50 lines** to maintain stable prefix
2. **Don't modify `CLAUDE.md` mid-session**—use `/new` to restart if changes needed
3. **Place `cache_control` breakpoints** at natural boundaries: system prompt, tool definitions, large user messages
4. **Use stable system prompts**—no timestamps, random IDs, or user-specific content in the prefix
5. **Batch 2-5 requests** within the cache TTL to amortize the write cost

5.3. Conversation Compaction

Definition

Compaction is the process of summarizing a long conversation to free context space while preserving essential information. Claude Code provides a `/compact` command for manual control and `auto-compact` at ~75-80% capacity.

`/compact` vs. `/clear` (Critical Distinction for Exam)

Dimension	<code>/compact</code>	<code>/clear</code> (aka <code>/new</code> , <code>/reset</code>)
What it does	Summarizes conversation, preserves compressed state	Wipes conversation entirely, fresh start
Memory preserved	Summary of completed work, modified files, current plan	Nothing— <code>CLAUDE.md</code> reloads but conversation is gone
When to use	Mid-task, at logical breakpoints	Switching to completely different task
Token recovery	Partial (summary still consumes 3K-5K tokens)	Complete (full context window available)
Risk	Lossy—summary may miss details	No risk if task is done; total loss if in-progress

Source:

Compaction API (Beta in Opus 4.6)

Anthropic introduced a **server-side compaction API** that summarizes context automatically without truncation. When context approaches the window limit, the API automatically summarizes earlier conversation parts instead of truncating them.

Customizing the Compact Prompt

Without specific instructions, `/compact` produces generic summaries like "investigated auth bug, modified files, ran tests"—which tells the agent almost nothing actionable.

Effective compact prompt examples:

Preserve specific debugging context

```
/compact preserve the stack trace from the auth error, all modified files in src/middlewares/, and the hypothesis that the token refresh is racing with the session check
```

Preserve refactoring progress

```
/compact preserve the list of files migrated from REST to GraphQL, the remaining files to migrate, and the schema changes in src/graphql/
```

Persistent Compact Instructions in CLAUDE.md

Add a "Compact Instructions" section to CLAUDE.md to apply rules to every compaction (including auto-compact):

```
# Compact Instructions
```

When compacting, always preserve:

- All modified file paths with the specific changes made
- Current test results (which pass, which fail, exact error messages)
- The current task and remaining TODO items
- Any architectural decisions made and their reasoning
- Active debugging hypotheses

The Compaction Feedback Loop Problem

Sessions that compact once tend to compact repeatedly. Each compaction forces the agent to re-read files to recover lost context, consuming more tokens, accelerating the next compaction.

Breaking the loop: The solution is not better compaction but less waste per operation. Reduce file read tokens by using targeted operations (not reading entire files) and selective rewrites.

Micro Compact (Automatic, Lighter)

When context usage reaches ~60-70%, Claude Code may run a **micro compact** pass that selectively clears older tool outputs while preserving the conversation flow.

Aspect	Micro Compact	Full /compact
Trigger	Automatic at ~60-70% context	Manual or auto at ~75-80%
What gets cleared	Old tool outputs only	Tool outputs + conversation summary
Conversation preserved	Fully intact	Summarized
Token recovery	Moderate (10-30K)	Large (50-150K)

5.4. Token Estimation and Budget Management

Context Window Anatomy

The 200K token window sounds large, but ~86K tokens are consumed before you type your first message:

Component	Typical Tokens	Notes
System prompt + tools	~20,000	Fixed cost, always present
MCP tool schemas	900-51,000	More MCP servers = faster compaction
CLAUDE.md files	300-2,000	Survives compaction, loads every request
Auto-compact buffer	~33,000	Reserved, not usable
Conversation + outputs	~114,000	This fills up and triggers compaction

Token Consumption Hotspots

Operation	Token Cost	Notes
File reads	2K-5K each	Reading entire files for 10-20 lines of need
File rewrites	200-5K each	Default edit pattern rewrites entire files
Conversation history	Accumulates per turn	Every turn re-sends full history
Tool outputs	Variable	grep, bash output, large JSON responses

Cost Control Strategies

For `/goal` autonomous mode (days-long runs):

- Token costs compound because each turn reprocesses full conversation history
- **No built-in spending caps tied to goal completion**—add custom conditions like turn limits
- Past incidents exceeded \$6,000 in a single session from unbounded loops

Control	Implementation	Purpose
--max-turns N	CLI flag	Cap iterations
Turn limit in goal condition	while turns < 50 and not goal_complete	Prevent infinite loops
External token monitoring	Custom script with Anthropic API	Real-time cost tracking
Checkpointing	Save state after each major action	Resume without reprocessing

Token Estimation Formula (Exam Reference)

For a multi-turn session:

Total tokens = Fixed_overhead + (Avg_tokens_per_turn × Number_of_turns)

Where Fixed_overhead = System_prompt + Tools + CLAUDE.md + Tool_schemas

And tokens_per_turn = Conversation_history + Current_input + Tool_outputs + Response

Exam Tip: Conversation history grows linearly with each turn. After 10+ turns, history dominates the budget.

We have 700 Practice test questions **Claude Certified Architect Foundations (CCA-F)** (Taken from previous exams)

Full Practice Set link below

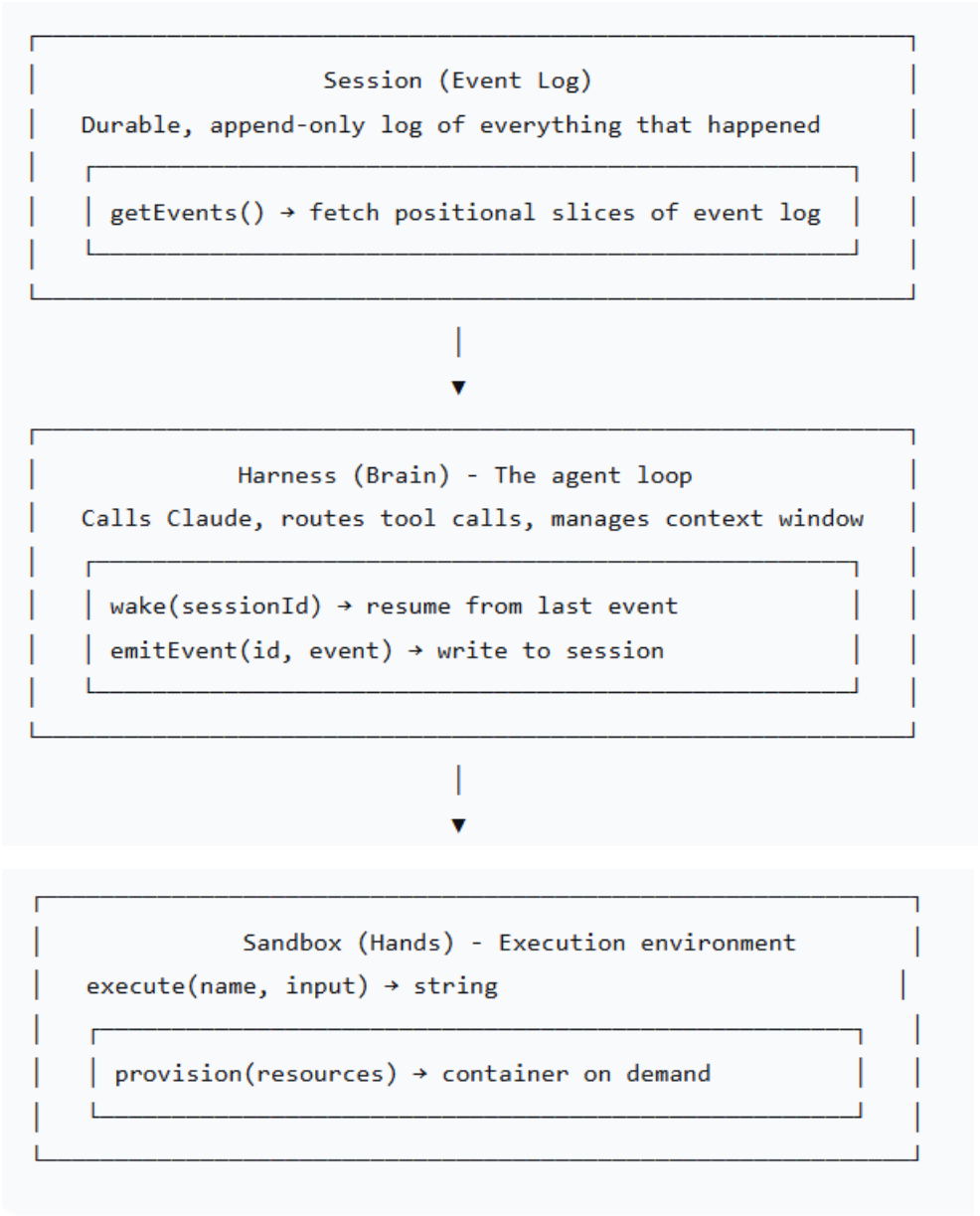
<https://skillcertpro.com/product/claude-certified-architect-foundations-cca-f-exam-questions/>

100% Money back Guarantee, If you don't pass the exam in 1st attempt, your money will be refunded back

5.5. Multi-Turn Conversation Design

The Session as Event Log (Managed Agents Architecture)

Anthropic's Managed Agents architecture separates three concerns:



The Key Insight: Session ≠ Context Window

Critical for exam: The session is the durable event log. Claude's context window is just a **view** into that log. Compaction, trimming, and transformation happen at the view level—the underlying session data is never lost.

Multi-Turn Design Patterns

Pattern	Implementation	Use Case
Checkpointing	Save state after each major action	Recovery from failures

Pattern	Implementation	Use Case
Idempotent operations	Same request → same result	Retry safety
Context trimming	Selective removal of old tool results	Keep window within limits
Separated auth	Tokens never reach sandbox	Security across turns

Security in Multi-Turn: The Vault Pattern

Credentials are never exposed to the sandbox where generated code runs:

- **Git pattern:** Clone repo using token during sandbox init; wire into local git remote. The agent executes git push without ever handling the token.
- **MCP proxy pattern:** OAuth tokens stored in secure vault. Claude calls MCP tools via proxy; proxy fetches credentials from vault using session ID.

Context Anxiety

Older models (Claude Sonnet 4.5) exhibited **context anxiety**—wrapping up tasks prematurely as the context limit approached. Opus 4.5 no longer shows this behaviour.

Exam Insight: Harnesses that worked around context anxiety became "dead weight" when models improved. Design interfaces, not model-specific workarounds.

Summary Table – Domain 5 Core Topics

Topic	Must-Know for Exam
CALM Framework	Three legs: autonomy, sandbox, legible progress; peripheral over interrupt
Prompt Caching	Prefix matching; 90% savings; cache_control breakpoints on stable content
Cache Traps	CLAUDE.md changes, dynamic timestamps, model switching, /compact
Compaction	/compact (mid-task, lossy summary) vs /clear (task switch, complete wipe)
Token Budget	~86K fixed overhead, ~114K available for work
Session vs. Window	Session = durable log; Context window = view
Managed Agents	Brain (harness) + Hands (sandbox) + Session (event log)

Key Takeaways (Exam Cram)

1. **CALM framework** prioritizes peripheral signals over interrupts—design agents you check when ready, not ones that tap your shoulder.
2. **Prompt caching uses prefix matching**—any change early in the prompt invalidates everything after it. Keep CLAUDE.md stable mid-session.
3. **Cache hit = 0.1× price, miss = 1.0× or 1.25×** —90% savings when configured correctly. Break-even is 1-2 hits.
4. **/compact vs /clear**: Use /compact mid-task to preserve summarized state; use /clear between unrelated tasks for a fresh start.
5. **The compaction feedback loop** happens when compacted summaries force re-reading files, consuming more tokens, accelerating next compaction.
6. **Session is not the context window**—session is durable storage; context window is a transformable view. This enables recovery without data loss.
7. **Separate brain from hands** for reliability: harness can crash and restart; session persists; sandbox can be recreated on demand.
8. **Never put credentials where generated code runs**—use vaults and proxies to keep tokens unreachable.

We have 700 Practice test questions **Claude Certified Architect Foundations (CCA-F)** (Taken from previous exams)

Full Practice Set link below

<https://skillcertpro.com/product/claude-certified-architect-foundations-cca-f-exam-questions/>

100% Money back Guarantee, If you don't pass the exam in 1st attempt, your money will be refunded back

Disclaimer: All data and information provided on this site is for informational purposes only. This site makes no representations as to accuracy, completeness, correctness, suitability, or validity of any information on this site & will not be liable for any errors, omissions, or delays in this information or any losses, injuries, or damages arising from its display or use. All information is provided on an as-is basis.